МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра автоматики и процессов управления

ОТЧЕТ

по лабораторной работе №9

по дисциплине «Программирование»

Тема: Работа со структурами.
Обработка массива структур STUDENT

Студент гр. 6391 _____ Рудой И.В.

Преподаватель _____ Писарев А.С.

Санкт-Петербург
2026

Цель работы (задание)

Описать структуру с именем STUDENT, содержащую следующие поля:

- ФИО;
- номер группы;
- успеваемость (массив из пяти оценок по 5-бальной шкале).

Реализовать программу, выполняющую следующие действия:

1. ввод с клавиатуры данных в массив, состоящий из заданного с клавиатуры количества структур типа STUDENT;
2. упорядочивание массива по возрастанию номера группы;
3. вывод на дисплей фамилий и номеров групп для всех студентов, включенных в массив, если средний балл студента больше 4,0 (если таких студентов нет, вывести соответствующее сообщение);
4. вывод на экран таблицы, содержащей столбцы: «Номер группы», «Количество студентов», «Количество двоечников». Строки таблицы должны быть упорядочены по убыванию значений в столбце «Количество двоечников».

Примечание: под двоечником понимается студент, средний балл которого меньше 3,0.

Алгоритм решения

Для хранения данных о студенте объявляется структура STUDENT с полями: fio (строка), group (целое), grades[5] (массив из пяти целых). Для сводки по группам объявляется вторая структура GroupStats с полями: groupNum, total, fails.

Массив студентов создаётся динамически: размер вводится с клавиатуры, память выделяется оператором new[]. Для ФИО используется getline, чтобы строка могла содержать пробелы. Перед первым getline и в конце каждой итерации ввода вызывается cin.ignore(10000, '\n') – это очищает буфер от символа перевода строки, оставшегося после числового ввода.

Алгоритм работы программы:

1. Ввод количества студентов n с проверкой n > 0.

2. Ввод данных каждого студента: ФИО одной строкой, номер группы, пять оценок.
3. Пункт б) – сортировка массива методом пузырька по возрастанию поля group. Обмен выполняется целыми структурами (STUDENT temp = students[j]), что копирует все поля сразу.
4. Пункт в) – проход по отсортированному массиву. Для каждого студента вычисляется средний балл функцией average(). Если он больше 4,0 – выводятся фамилия и номер группы. Счётчик found отслеживает, найден ли хотя бы один такой студент; если нет – выводится сообщение.
5. Пункт г) – группировка. Так как массив уже отсортирован по группе, все студенты одной группы идут подряд. За один проход по массиву для каждой группы подсчитывается общее число студентов и число двоечников (средний балл < 3,0). Данные сохраняются в массив stats. Затем stats сортируется методом пузырька по убыванию поля fails. Таблица выводится через printf с фиксированной шириной столбцов.
6. Освобождение памяти операторами delete[].

Текст программы

```
#include <iostream>
#include <cstdio>
#include <string>

using namespace std;

//1. Описать структуру с именем STUDENT, содержащую следующие поля :
//• ФИО;
//• номер группы;
//• успеваемость(массив из пяти оценок по 5 - бальной шкале).
//2. Реализовать программу, выполняющую следующие действия :
//а) ввод с клавиатуры данных в массив, состоящий из заданного с клавиатуры
//количества структур типа STUDENT;
//б) упорядочивание массива по возрастанию номера группы;
//в) вывод на дисплей фамилий и номеров групп для всех студентов, включенных
в
//массив, если средний балл студента больше 4, 0 (если таких студентов нет,
вывести
//соответствующее сообщение);
//г) вывод на экран таблицы, содержащей столбцы : «Номер группы», «Количество
//студентов», «Количество двоечников».Строки таблицы должны быть упорядочены
по
//убыванию значений в столбце «Количество двоечников».
```

```
struct STUDENT {
    string fio;          // ФИО одной строкой
    int group = 0;
    int grades[5] = { 0,0,0,0,0 };
};

struct GroupStats {
    int groupNum;
    int total;
    int fails;
};

double average(STUDENT s) {
    double sum = 0.0;
    for (int i = 0; i < 5; i++) {
        sum += s.grades[i];
    }
    return sum / 5.0;
}

int main()
{
    int n;
    cout << "Введите количество студентов:" << endl;
    cin >> n;
    //Проверка, что количество студентов больше 0
    if (n <= 0) {
        cout << "Количество студентов должно быть положительным." << endl;
        return 1;
    }
    cin.ignore(10000, '\n');

    //Ввод массива структур
    cout << "а) Ввод с клавиатуры данных в массив:" << endl;
    STUDENT* students = new STUDENT[n];
    for (int i = 0; i < n; i++) {
        cout << "Введите ФИО студента:" << endl;
        getline(cin, students[i].fio);

        cout << "Введите номер группы:" << endl;
        cin >> students[i].group;

        cout << "Введите 5 оценок:" << endl;
        for (int j = 0; j < 5; j++) {
            cin >> students[i].grades[j];
        }
        cin.ignore(10000, '\n');
    }

    cout << "б) Упорядочивание массива по возрастанию номера группы." <<
endl;
    //Сортировка пузыркем
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1 - i; j++) {
```

```
            if (students[j].group > students[j + 1].group) {
                STUDENT temp = students[j];
                students[j] = students[j + 1];
                students[j + 1] = temp;
            }
        }
    }
    cout << "в) Вывод на дисплей фамилий и номеров групп, если средний балл
студента больше 4:" << endl;
    int found = 0;
    for (int i = 0; i < n; i++) {
        if (average(students[i]) > 4.0) {
            cout << students[i].fio << endl;
            cout << students[i].group << endl;
            found++;
        }
    }
    if (found == 0) {
        cout << "Нет студентов со средним баллом > 4,0" << endl;
    }

    cout << "г) Вывод на экран таблицы:" << endl;
    GroupStats* stats = new GroupStats[n];
    int groupCount = 0;

    for (int i = 0; i < n; i++) {
        //Группа сменилась – создаём новую карточку
        if (groupCount == 0 || stats[groupCount - 1].groupNum !=
students[i].group) {
            stats[groupCount].groupNum = students[i].group;
            stats[groupCount].total = 0;
            stats[groupCount].fails = 0;
            groupCount++;
        }

        //Обновляем счётчики последней карточки
        //Двоечники - это те, у кого средний балл ниже 3
        stats[groupCount - 1].total++;
        if (average(students[i]) < 3.0) {
            stats[groupCount - 1].fails++;
        }
    }

    //Сортировка массива stats по убыванию
    for (int i = 0; i < groupCount - 1; i++) {
        for (int j = 0; j < groupCount - 1 - i; j++) {
            if (stats[j].fails < stats[j + 1].fails) {
                GroupStats temp = stats[j];
                stats[j] = stats[j + 1];
                stats[j + 1] = temp;
            }
        }
    }
```

```
    printf("%15s %20s %20s\n", "Номер группы", "Количество студентов",
"Количество двоечников");
    printf("-------------------------------------------------------\n");

    for (int i = 0; i < groupCount; i++) {
        printf("%12d %20d %20d\n",
            stats[i].groupNum,
            stats[i].total,
            stats[i].fails);
    }

    //Освобождение памяти
    delete[] students;
    delete[] stats;

    return 0;
}
```

Описание тестов

Для тестирования используются данные с фамилиями, записанными латиницей, – в связи с особенностью отображения кириллицы в консоли Visual Studio при отладке. Под двоечником понимается студент со средним баллом меньше 3,0.

Тест 1. Смешанный набор (базовый случай)

Вход:

5

Ivanov Ivan Ivanovich

6391

5 5 5 5 5

Petrov Petr Petrovich

6391

2 2 2 2 2

Sidorov Sidor Sidorovich

6392

4 4 4 4 4

Kozlov Kozel Kozlovich

6392

2 3 2 3 2

Smirnov Smirn Smirnovich

6391

5 4 5 4 5

Ожидаемый выход:

а) Ввод с клавиатуры данных в массив:

Введите ФИО студента:

Ivanov Ivan Ivanovich

...

б) Упорядочивание массива по возрастанию номера группы.

в) Вывод на дисплей фамилий и номеров групп, если средний балл студента больше 4:

Ivanov Ivan Ivanovich

6391

Smirnov Smirn Smirnovich

6391

г) Вывод на экран таблицы:

Номер группы	Количество студентов	Количество двоечников
6391	3	1
6392	2	1

Тест 2. Все средние баллы больше 4,0

Вход:

3

Ivanov Ivan Ivanovich

6392

5 5 5 5 5

Petrov Petr Petrovich

6391

5 5 4 5 5

Sidorov Sidor Sidorovich

6392

4 5 4 5 5

Ожидаемый выход:

а) Ввод с клавиатуры данных в массив:

Введите ФИО студента:

Ivanov Ivan Ivanovich

...

б) Упорядочивание массива по возрастанию номера группы.

в) Вывод на дисплей фамилий и номеров групп, если средний балл студента больше 4:

Petrov Petr Petrovich

6391

Ivanov Ivan Ivanovich

6392

Sidorov Sidor Sidorovich

6392

г) Вывод на экран таблицы:

Номер группы	Количество студентов	Количество двоечников
6391	1	0
6392	2	0

Тест 3. Нет студентов со средним баллом выше 4,0

Вход:

4

Ivanov Ivan Ivanovich

6391

3 3 3 3 3

Petrov Petr Petrovich

6392

2 2 2 2 2

Sidorov Sidor Sidorovich

6391

3 4 3 4 3

Kozlov Kozel Kozlovich

6392

2 3 3 3 2

Ожидаемый выход:

а) Ввод с клавиатуры данных в массив:

Введите ФИО студента:

Ivanov Ivan Ivanovich

...

б) Упорядочивание массива по возрастанию номера группы.

в) Вывод на дисплей фамилий и номеров групп, если средний балл студента больше 4:

Нет студентов со средним баллом > 4,0

г) Вывод на экран таблицы:

Номер группы	Количество студентов	Количество двоечников
6392	2	2
6391	2	0

Тест 4. Сортировка таблицы по убыванию двоечников (смена порядка)

Вход:

4

Ivanov Ivan Ivanovich

6391

2 2 2 2 2

Petrov Petr Petrovich

6392

2 2 2 2 2

Sidorov Sidor Sidorovich

6391

5 5 5 5 5

Kozlov Kozel Kozlovich

6392

2 2 2 2 2

Ожидаемый выход:

а) Ввод с клавиатуры данных в массив:

Введите ФИО студента:

Ivanov Ivan Ivanovich

...

б) Упорядочивание массива по возрастанию номера группы.

в) Вывод на дисплей фамилий и номеров групп, если средний балл студента больше 4:

Sidorov Sidor Sidorovich

6391

г) Вывод на экран таблицы:

Номер группы	Количество студентов	Количество двоечников
6392	2	2
6391	2	1

Тест 5. Одна группа (граничный случай)

Вход:

3

Ivanov Ivan Ivanovich

6391

5 5 5 5 5

Petrov Petr Petrovich

6391

3 3 3 3 3

Sidorov Sidor Sidorovich

6391

2 2 2 2 2

Ожидаемый выход:

а) Ввод с клавиатуры данных в массив:

Введите ФИО студента:

Ivanov Ivan Ivanovich

...

б) Упорядочивание массива по возрастанию номера группы.

в) Вывод на дисплей фамилий и номеров групп, если средний балл студента больше 4:

Ivanov Ivan Ivanovich

6391

г) Вывод на экран таблицы:

Номер группы	Количество студентов	Количество двоечников
6391	3	1

Тест 6. Проверка некорректного количества студентов

Вход:

0

Ожидаемый выход:

Введите количество студентов:

0

Количество студентов должно быть положительным.

Пояснение: при вводе n = 0 срабатывает проверка if (n <= 0) в начале main. Программа выводит сообщение об ошибке и завершается с кодом возврата 1, не переходя к вводу данных о студентах. Тест проверяет корректность обработки граничного значения – количество студентов должно быть строго положительным.

Результат работы

Ниже приведены скриншоты консоли, подтверждающие корректную работу программы на разработанных тестах.

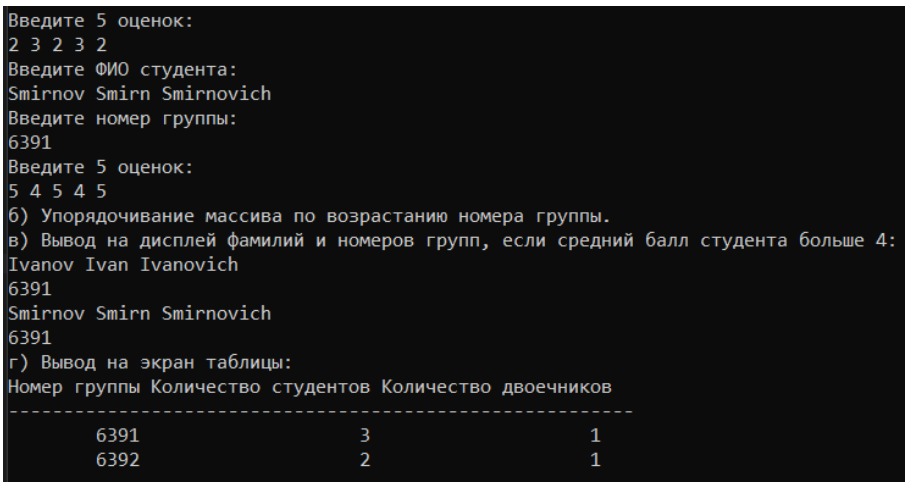


Рисунок 1. Результаты выполнения теста 1

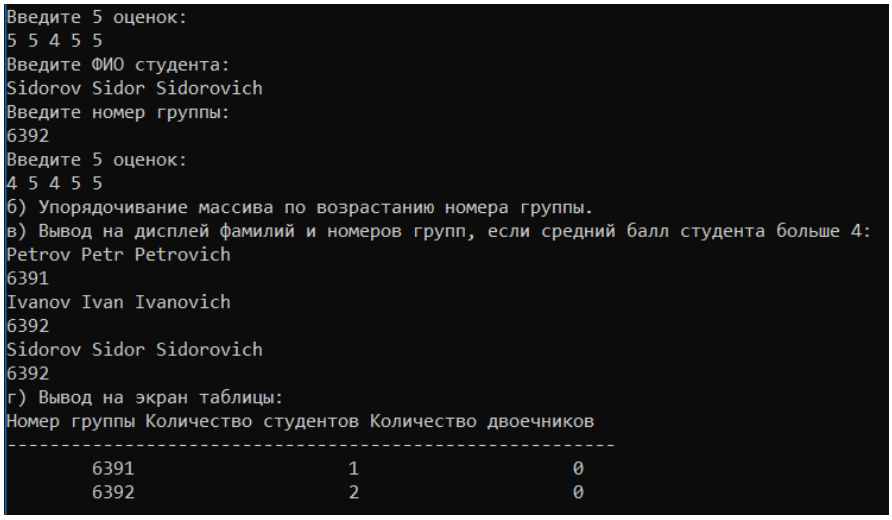


Рисунок 2. Результаты выполнения теста 2

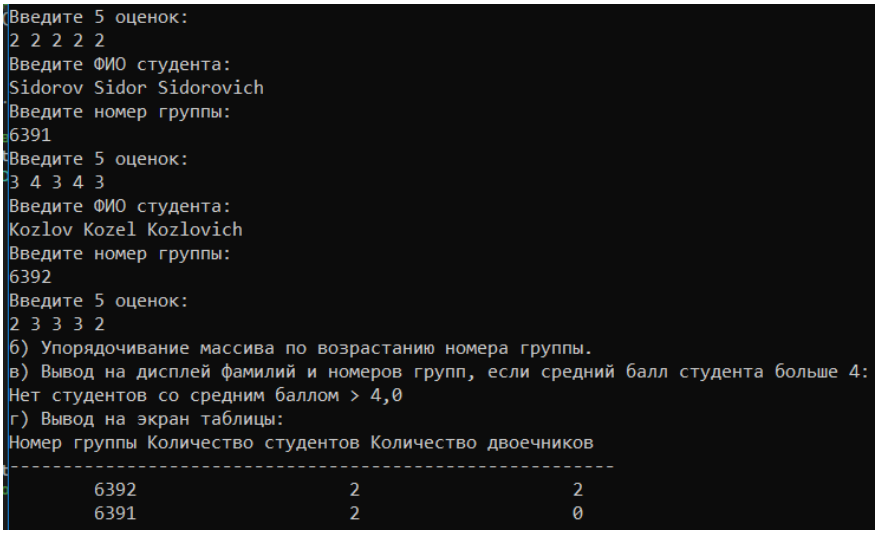


Рисунок 3. Результаты выполнения теста 3

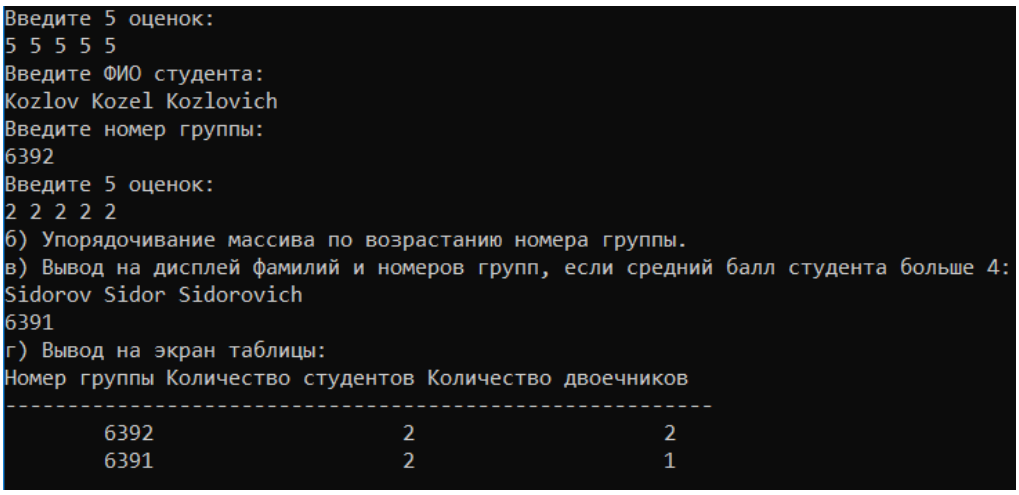


Рисунок 4. Результаты выполнения теста 4

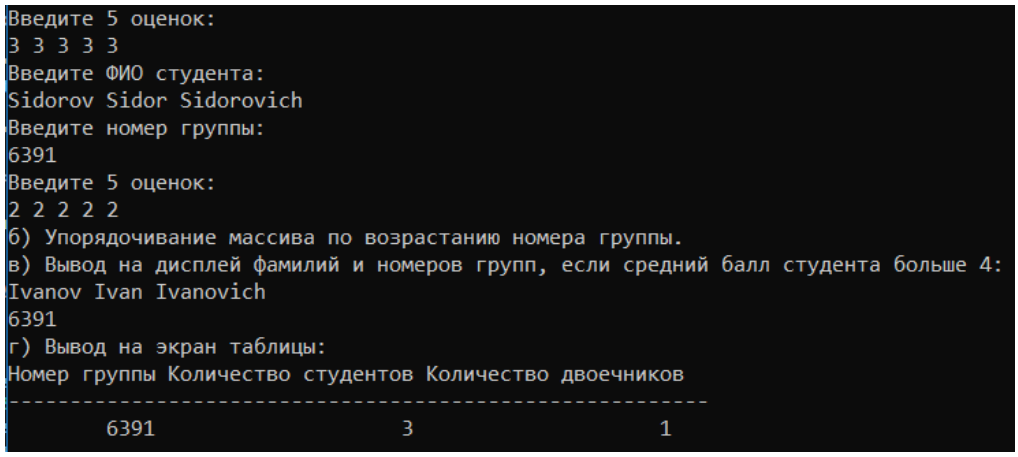


Рисунок 5. Результаты выполнения теста 5

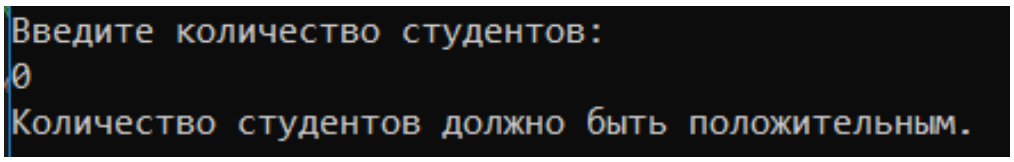


Рисунок 6. Результаты выполнения теста 6

Все тесты отработали в соответствии с ожидаемыми результатами. Программа корректно выполняет все четыре пункта задания: вводит данные о студентах, сортирует массив по возрастанию номера группы, выводит студентов со средним баллом выше 4,0, формирует таблицу групп с количеством студентов и двоечников, упорядоченную по убыванию числа двоечников.

Вывод

В ходе лабораторной работы была разработана программа на языке C++, обрабатывающая массив структур типа STUDENT.

Программа реализует все требуемые действия:

1. ввод с клавиатуры данных в динамический массив структур заданного размера;
2. сортировку массива по возрастанию номера группы методом пузырька;
3. вывод фамилий и номеров групп для студентов со средним баллом выше 4,0, а также сообщение при отсутствии таких студентов;
4. построение таблицы с колонками «Номер группы», «Количество студентов», «Количество двоечников», упорядоченной по убыванию количества двоечников.

В процессе работы закреплены навыки:

- объявления структур с полями разных типов;
- работы с динамической памятью (new[] / delete[]);
- ввода строк с пробелами через getline с корректной очисткой буфера;
- сортировки массива структур с обменом целых структур;
- группировки данных за один проход по отсортированному массиву;
- использования второй структуры для сводной информации;
- форматированного вывода таблицы через printf.

Тестирование на шести наборах данных подтвердило корректность работы программы, включая граничные случаи: отсутствие подходящих студентов по среднему баллу, одну группу, а также смену порядка строк таблицы при сортировке по убыванию двоечников.